Jev-as-IF: Graph-Guided Compilation of Agent Tool Chains for the Web

Anonymous Author(s)

Abstract

Web agents act through chains of tool calls over REST and Model Context Protocol (MCP) services. In a ReAct loop, every dependent call costs an LLM round over a growing context. Letting the agent write code removes these rounds only for mechanical steps, such as looking up a field or computing a value. Semantic steps still send control back to the model: deciding which returned entity the user meant, which branch applies, or which items to keep. We present Jev-as-IF, a *graph-guided tool compiler*. The agent writes the whole tool chain once as a program, and the chain then runs to completion without returning to the LLM.

The compiler rests on two designs. First, it reads a *split object–operation graph* (SOG). The object graph describes business entities, properties, relations and rules. The operation graph describes tool-backed operations, and its data-flow edges are derived from which operations produce and consume each property. From the SOG, the compiler renders typed stubs for the agent, statically checks the agent's program, and obtains the entity semantics that run-time decisions need. Second, every semantic decision in the program is written as an explicit conditional on a System-1 primitive (`jev.choice`, `jev.noul`). These conditionals are evaluated at run time by a calibrated non-generative model (Jev), and a confidence gate hands a decision back to the LLM only when it is uncertain.

The agent therefore writes code once, the compiler checks and optimizes it, and Jev evaluates its decisions. The LLM is called to plan, to answer, and again only when a decision is uncertain. **[TODO:** Headline results on FinMCP-Bench vs. ReAct / ReWOO / LLMCompiler / programmatic tool calling: LLM calls, tokens, latency at no loss of success; reduction of invalid-call and wrong-entity errors; main effects of graph guidance and Jev-as-IF.**]**

CCS Concepts

• **Information systems** → **Web applications**; • **Computing methodologies** → *Planning and scheduling*.

Keywords

LLM agents, tool calling, Model Context Protocol, programmatic tool calling, compilation, ontology, knowledge graph, System-1 models, calibration

ACM Reference Format:

Anonymous Author(s). 2027. Jev-as-IF: Graph-Guided Compilation of Agent Tool Chains for the Web. In *Proceedings of the ACM Web Conference 2027 (WWW '27), May 10–14, 2027, Dublin, Ireland*. ACM, New York, NY, USA, 8 pages.

1 Introduction

LLM agents now reach the Web through tool protocols. Wealth-management assistants, travel agents and CRM copilots all expose REST services as tools, increasingly through the Model Context Protocol (MCP) [3], and let a language model decide which tools to call and with which arguments [19, 32]. How these services are *described* to the agent and how the resulting calls are *executed* are therefore questions of Web infrastructure. Together they decide the latency, cost and reliability of every agent built on these services.

Tool calls are rarely independent. Answering "how much does the steady portfolio I hold charge?" requires listing the customer's portfolios, deciding which one the customer means, reading its fee-schedule identifier, and only then querying the fee schedule. In a ReAct loop [32], each dependent step costs one LLM round, and every round re-reads all previous tool outputs. On the multi-tool split of FinMCP-Bench [37], reference trajectories take **[TODO:** 5.7**]** LLM rounds, and the median task returns about **[TODO:** 38k**]** characters of tool output.

The strongest current answer is to let the agent *write code*. CodeAct [30] and programmatic tool calling (PTC) [4] have the model write one program that chains the calls in a sandbox, keeping intermediate results out of the context. Plan-then-execute compilers [12, 31] do the same with a plan language. These approaches remove an LLM round only when the step is *mechanical*: a field lookup, a filter on a numeric field, a computation. Two obstacles remain.

Semantic decisions return control to the model. At some steps the next argument or branch depends on a judgment over data that only exists at run time: "the steady one" among twelve returned portfolios, "the announcements that could explain the drop", whether a product's risk text matches the customer's concern. A program cannot make this judgment. It can hard-code a brittle string match, or it can return the data to the model and pay for another round. Our screening of FinMCP-Bench (§6.1) finds such decisions in most multi-tool tasks.

Code over unfamiliar tools is written blind. MCP tools usually declare inputs but not outputs, cross-tool dependencies or business rules. A model writing code against them must guess field names, or first print a result and look at it, which again adds rounds. Nothing checks the program before it runs, so invalid calls and wrong arguments surface only as run-time failures.

We present Jev-as-IF, a *graph-guided tool compiler* that addresses both obstacles. The agent writes the tool chain once, in a restricted Python subset (PyIF), against typed stubs that the compiler derives from a domain graph. Every semantic decision is written as a call to a System-1 primitive, such as `if jev.noul(...):` or `p = jev.choice(...)`. This is where the name comes from: a System-1 model is the evaluator of the program's `if`s. The compiler type-checks the program against the graph, turns each primitive into a well-posed question using the graph's entity semantics, batches and parallelizes, and lowers the program to an executable flow. At run time, tools are executed by the runtime and decisions are evaluated

WWW '27, Dublin, Ireland
2027.

by Jev [28], a calibrated non-generative model. A confidence gate returns a decision to the LLM only when it is uncertain.

Two designs make this work.

- **A split object–operation graph (SOG) as the compiler's input.** The object graph models entities, properties, relations and business rules. The operation graph models tool-backed operations, and its data-flow edges are derived from property production and consumption, so nobody annotates them by hand. Existing graph-guided agents use a graph to *select or plan* tools [13, 25, 26]. Jev-as-IF uses the SOG to *compile*. The operation graph supplies types and data-flow checks; the object graph supplies the labels, meanings and rules that make each run-time decision well posed.
- **Jev-as-IF: System-1 models as the evaluators of program conditionals.** Choosing among a few typed candidates, checking a yes/no guard, or keeping items in a filter needs neither reasoning traces nor generated text. It needs a fast closed-set judgment with an honest probability. Placing these judgments *inside* the program as typed primitives is what lets a chain with semantic steps run end to end. Conformal calibration bounds the probability that the program acts on a wrong decision.

The result is fast and cheap by construction: the LLM is called to plan and to answer, and otherwise only when a decision is uncertain. It does not trade away accuracy. Static checking removes invalid calls before execution, graph semantics reduce wrong-entity decisions, and provenance checks reject fabricated arguments.

Contributions.

- **Jev-as-IF, a graph-guided tool compiler** for agent tool chains: the PyIF language with System-1 primitives, graph-based static checking, elaboration of decisions with entity semantics, batching and parallelization, and lowering to an executable flow with confidence-gated fallback (§4).
- **Split object–operation graphs** as the compiler's input, with derived data flow, object-scoped disclosure, rendering of typed stubs, and a construction procedure from tool schemas and call logs (§3).
- **An evaluation** on FinMCP-Bench, screened into tasks with and without semantic decisions, together with an anonymized advisory deployment and synthetic scaling suites. A factorial study on top of programmatic tool calling (graph guidance × Jev-as-IF) and targeted ablations isolate each design (§6). **[TODO:** release code, graphs and screened splits]

2 Background and Motivation

2.1 Dependent Tool Chains on the Web

An agent-ready Web service exposes operations with typed inputs and JSON outputs. With MCP [3], an agent discovers these operations at run time and calls them through a uniform protocol. A request is typically served by a chain of calls in which later inputs depend on earlier outputs. In a ReAct loop every dependent call costs a full LLM round, and the model re-reads the system prompt, the tool definitions and all previous observations each time. Cost therefore grows roughly quadratically with chain length. Code-writing agents collapse the chain into one program, but only as far as each step can be written as code.

Table 1: How a step obtains what it needs. Jev-as-IF gives every kind except generation an executor that does not call the LLM.

Kind	Example	Executor
Constant	date range stated in the request	program literal
Reference	fee code of the selected portfolio	program (field access)
Transform	days between two dates	program (code)
Semantic decision	which of 12 portfolios is "the steady one"; which announcements could explain a drop	**System-1 primitive** (Jev-as-IF)
Generation	the final explanation to the user	LLM

2.2 Which Steps Need the Model?

Table 1 classifies how a step obtains what it needs from earlier results. Constants, references and transformations can be written in code. Generation needs a language model and belongs at the end of the chain, where the answer is written. The remaining kind is the *semantic decision*: a judgment over a closed set that only materializes at run time. It covers choosing among returned entities, choosing a branch, keeping or dropping items, and checking a qualitative precondition. Code cannot express such a decision, and an LLM is far more than it needs. A semantic decision needs a closed-set judgment with a calibrated probability.

2.3 System-1 Models

Non-generative "System-1" models such as Jev [28] answer three typed question forms: *Choice* (one of up to 255 options, with a probability per option), *Noul* (probability of yes) and *Score* (ordered levels). They answer in tens to hundreds of milliseconds, are priced per input token, and charge nothing for output. Many questions over the same state can be answered in one request. An independent evaluation [14] found that Jev's confidence orders its errors: accuracy is 47.7% when the top probability is below 0.6 and 99.1% when it is at least 0.99. The same study also reports sensitivity to option order and confident errors on adversarially styled inputs. Jev's documentation advises keeping arithmetic and date comparison in code, pruning irrelevant state, and stating decision criteria explicitly [28]. These are exactly the properties an evaluator of program conditionals needs, together with the weaknesses the compiler must guard against.

3 The Graph: Split Object–Operation Graphs

The compiler reads one artifact per tool domain, a split object–operation graph. This section defines it, explains why it is split, and shows how the compiler presents it to the agent and how it is built.

Table 2: What each representation gives the agent and the compiler.

Representation	Output types	Data flow	Entity semantics	Scoped disclosure	Separate upkeep
Flat tool schemas	–	–	–	–	✓
Tool dependency graph	✓	annotated	–	–	–
Knowledge graph only	–	–	✓	✓	–
Unified single graph	✓	mixed	✓	partial	–
SOG (ours)	✓	**derived**	✓	✓	✓

3.1 Definition

A SOG is a pair $(G_{\mathrm{O}}, G_{\mathrm{A}})$ coupled only through properties.

Object graph. $G_{\mathrm{O}} = (\mathcal{E}, \mathcal{P}, \mathcal{R})$ consists of entity types $\mathcal{E}$ (e.g., Customer, Portfolio, FeeSchedule), properties $\mathcal{P}$ written Entity.prop, and directed relations $\mathcal{R} \subseteq \mathcal{E} \times \mathcal{E}$ (e.g., Portfolio→FeeSchedule, "is charged by"). Every entity and property carries a natural-language meaning, and entities may carry business rules ("a standard fee rate is not the customer's actual charge"). An identifier property may name a *label property* of the same entity (Portfolio.code is labeled by Portfolio.name).

Operation graph. $G_{\mathrm{A}} = (\mathcal{A}, D, F)$ consists of operations $\mathcal{A}$, each bound to a Web tool and declaring $\mathrm{in}(a), \mathrm{out}(a) \subseteq \mathcal{P}$. Data-flow edges are *derived*, never drawn by hand:

$$D = \{(a, a', p) \ : \ p \in \mathrm{out}(a) \cap \mathrm{in}(a')\}. \tag{1}$$

Optional control-flow edges F encode prescribed business processes (approval, signing). Otherwise $F = \emptyset$ and the agent composes operations freely.

Coupling. The only links between the graphs are property references. Each operation is *owned* by the entities it produces, $\mathrm{own}(a) = \{e : \exists\, e.p \in \mathrm{out}(a)\}$. An access annotation on each input property states who fills it (run identity, trusted host, agent, constant) and how it is checked (customer scope, provenance).

3.2 Why Split

Table 2 compares the SOG with the representations an agent typically receives.

- *Two kinds of edges, two jobs.* Object relations are conceptual: they say what is related to what, and they drive disclosure. Operation edges concern execution: they say what feeds what, and they drive type and data-flow checking. A single graph mixes the two, and an agent can mistake a conceptual relation for a call order.
- *Derived, checkable data flow.* Dependencies follow from property declarations, so nobody annotates tool-to-tool edges [13], and the compiler can verify that every input is produced upstream.
- *Entity semantics for run-time decisions.* A decision over raw codes is guesswork. The object graph supplies labels, meanings and rules that turn it into a well-posed question (§4.3).
- *Separate maintenance.* Entity models change slowly and belong to domain experts. Tool bindings change with services. Only property references must stay consistent, and they are checked automatically.

3.3 What the Agent Sees: Scoped, Typed Stubs

The agent never sees the whole tool set. Starting from an entry entity, the compiler takes the entities within h relation hops, S_h, and discloses an operation iff $\mathrm{own}(a) \subseteq S_h$. Inputs do not count, because nearly every operation consumes the customer and would otherwise always be shown. Entities just outside the scope are listed with their relations and owning operations, so the agent can request them. The agent's context is therefore bounded by the size of the object scope, not by the number of tools.

The disclosed part is rendered as PyIF stubs. Each entity becomes a record type whose fields are its properties, with their meanings as comments. Each operation becomes a function whose agent-filled parameters are typed and whose return type is a list of entity records. Identity- and host-filled parameters are omitted entirely, so the agent cannot even write a call on behalf of another customer. Entity and operation rules become docstrings. Appendix A shows an example.

3.4 Construction

We build a SOG from tool schemas and, when available, call logs, in six steps.

(1) *Operations*: one per tool; inputs from the schema; outputs from the response schema or, when that is missing, from fields observed in logged responses.
(2) *Property alignment*: merge fields that denote the same value across tools (e.g., `fundCode` and `code`), using name similarity and value co-occurrence in logs.
(3) *Entity induction*: group properties that co-occur in the same response records around an identifier; choose a label property (the most name-like co-occurring string field).
(4) *Relation induction*: add $e \longrightarrow e'$ when records of e carry identifiers of e'.
(5) *Derivation and validation*: derive D; check that every entity is reachable from the entry entity and every agent-filled input is producible or constant.
(6) *Curation*: an expert edits meanings, rules and access annotations.

Steps 1–5 are automatic. We report the curation effort and compare automatic and curated graphs in §6.3. **[TODO:** coverage metric: fraction of dependencies in reference trajectories recovered by derived edges.]

4 Jev-as-IF: Graph-Guided Tool Compilation

Figure 1 shows the pipeline. The agent writes a PyIF program against the disclosed stubs (§4.1). The compiler checks it against the operation graph (§4.2), elaborates its System-1 primitives with the object graph (§4.3), optimizes and lowers it (§4.4), and the runtime executes it under a confidence gate (§4.5).

4.1 The PyIF Language

PyIF is a restricted subset of Python. LLMs write Python fluently, and a direct comparison with programmatic tool calling requires the same medium. The subset is small enough that the compiler can analyze every program completely.

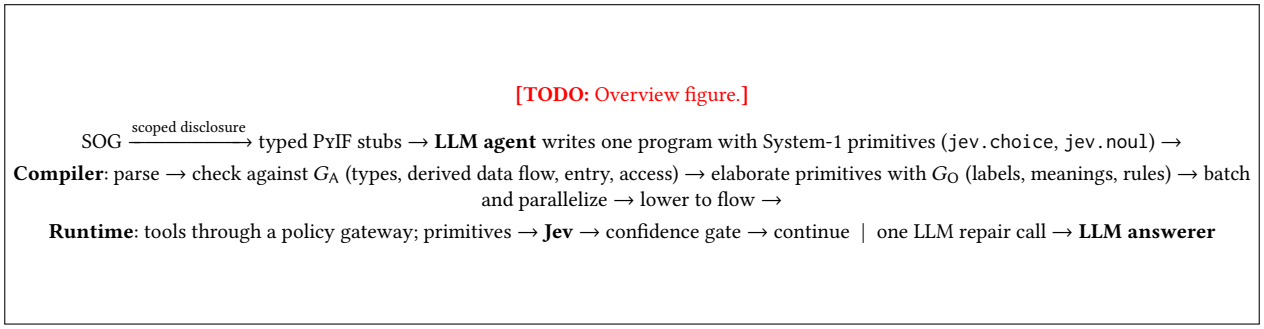


Figure 1: Jev-as-IF. The graph guides every stage: the operation graph types and checks the program, the object graph makes each decision well posed. The agent writes code once; Jev evaluates its decisions.

Allowed. Assignments; calls to disclosed operation stubs; field and index access on their results; `for` loops and comprehensions over tool results (bounded by result size); `if`/`else`; whitelisted pure built-ins (`len`, `sorted`, `min`, `max`, `sum`, date arithmetic); and a final `answer(...)` that names the facts to pass to the answerer.

Forbidden. Imports, `while`, recursion, user-defined functions, I/O, `eval`/`exec`, and string formatting of identifiers. These constructs are either unbounded or let the program fabricate an argument instead of obtaining it from a tool.

System-1 primitives. Semantic decisions are written as calls to three primitives. Each is typed by the graph.

- **`jev.choice(q, xs, k=1)`** returns the element of `xs` that answers question `q`; with `k="*"` it returns a sublist. `xs` must be a list of entity records or identifiers obtained from a tool. The result can bind later arguments.
- **`jev.noul(q, x)`** returns whether `q` holds for record `x` (or for the request). It is used as `if jev.noul(...):`, which is the literal meaning of *Jev-as-IF*.
- **`jev.filter(q, xs)`** keeps the elements of `xs` for which `q` holds.

The `jev` module is bound to a System-1 backend. Jev is our main instance, and we also evaluate open-source backends and an LLM backend (§6.4).

4.2 Graph-Guided Static Checking

The compiler parses the program into an abstract syntax tree, infers types from the stubs, and checks the following before any tool runs.

(1) *Language*: only allowed constructs appear.
(2) *Operations*: every call targets a disclosed operation with correctly typed arguments.
(3) *Data flow*: every field access refers to a property in out of the producing operation, consistent with the derived edges D. This rules out guessed field names.
(4) *Access*: no identity- or host-filled parameter is passed. Parameters that require provenance are bound to values derived from tool results or from primitives over tool results, never to literals.
(5) *Primitives*: candidate lists come from tool results, and questions refer only to fields in scope.
(6) *Order*: scope-producing entry operations precede scope-checked ones.

Violations go back to the agent as structured errors that name the line and the expected type. One repair round fixes them, and no tool has run yet.

4.3 Elaborating Primitives with the Object Graph

Each primitive call becomes one System-1 question. The object graph supplies what the agent's short question lacks.

- *Options*: for `jev.choice`, the candidate identifiers, each displayed through its entity's label property ("ZH0007 — Global Allocation QDII"), plus an abstain option $\perp$ ("none of these / cannot tell").
- *Criteria*: the meanings of the properties the question refers to, and the entity's rules as boundary cases.
- *State*: the user request plus only the referenced field values, because irrelevant fields degrade System-1 accuracy [28].

`jev.choice` with `k=1` becomes a Choice. `jev.choice` with `k="*"`, `jev.filter` and `jev.noul` become one Noul per candidate. Candidate sets larger than 255 are split into two levels.

4.4 Optimization and Lowering

The checked program is lowered to an executable flow. Calls become action nodes, primitives become selection nodes, `if` on a primitive becomes a pair of guarded edges, and loops become `foreach` nodes. Two optimizations follow from the dependency analysis. Independent calls run in parallel. Primitives that become ready in the same round and share state are *batched* into one System-1 request (speculative fan-out). Optionally, choice probabilities are averaged over r cyclic permutations of the options to reduce order sensitivity [14]. A value chosen by a primitive is by construction an observed value, so it passes provenance checks in the policy gateway.

4.5 Confidence-Gated Fallback

A decision is acted on only if the System-1 answer is confident enough. Let $\hat{p}(c \mid x)$ be the backend's probability for option c. On a calibration set of resolved decisions with nonconformity $1 - \hat{p}(y_i \mid x_i)$, let $\hat{\tau}$ be the $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile. The prediction set is $\Gamma(x) = \{c : 1 - \hat{p}(c \mid x) \le \hat{\tau}\}$, and the runtime acts only if

$\Gamma(x) = \{c\}$ with $c \neq \bot$. Under exchangeability, split conformal prediction [1, 29] gives $\Pr[y \in \Gamma(x)] \geq 1 - \alpha$. Acting on a wrong decision therefore has probability at most α, as in KnowNo [18]. A program with m primitives that uses α/m per primitive obeys a task-level bound of α. When the gate refuses, the runtime pauses and makes *one* LLM repair call. The call carries the partial trace and the option probabilities, and the LLM can decide, rewrite the rest of the program, or ask the user. **[TODO:** selective-risk variant [2, 9]]

4.6 Cost

Let C_0 be the agent prompt (request plus disclosed stubs) and $|o_j|$ the size of the j-th tool output. ReAct with L calls processes about $\sum_{i=1}^{L+1}\left(C_0 + \sum_{j<i}|o_j|\right)$ LLM input tokens. Programmatic tool calling saves the rounds for mechanical steps, but pays one round for every semantic decision whose data it must return to the model. Jev-as-IF processes C_0 to write the program and $C_0 + |o_{\text{ans}}|$ to answer, where o_{ans} holds only the facts named in `answer(...)`. It then adds $\Pr[\text{refuse}]$ repair calls and System-1 tokens, which are priced about two orders of magnitude lower. Object-scoped disclosure keeps C_0 bounded as the number of tools grows. **[TODO:** measured constants; predicted vs. measured.]

5 Implementation

We implement Jev-as-IF on an industrial ontology runtime for a wealth-management advisory service **[TODO:** anonymized description]. A SOG is stored as one document with an object part and an operation part. The runtime serves it through three MCP endpoints: `open` binds an authenticated customer and returns the disclosed scope, `query` discloses more of the object graph, and `execute` submits a decision.

The runtime supports both uses of the operation graph. In *object-led* use ($F = \emptyset$), an agent invokes operations freely under access checks. In *operation-led* use, a flow engine advances a control-flow graph (actions, selections, generation, fork/join, foreach, waits, sub-flows, compensation) and stops only where external input is needed. The compiler lowers a checked PyIF program into a transient operation-led flow. Its selection tasks, which an agent normally answers, are answered by the System-1 backend instead. Every write passes a single submission path with leases, idempotency keys, optimistic concurrency and an audit log, so compiled programs inherit concurrency, timeouts, retries and replay without new execution code.

The compiler front end uses Python's `ast` module. **[TODO:** lines of code for the graph builder, stub renderer, checker, elaborator and lowering; for benchmark experiments, whether lowering targets the runtime or a standalone executor.]

Backends. **[TODO:** pinned Jev version; open-source backend, e.g., a multilingual cross-encoder re-ranker or a small LM scored by option log-probabilities; agent and answerer LLMs]

Effort. **[TODO:** curation effort per SOG in person-hours and in numbers of entities, properties and operations]

6 Evaluation

We ask four questions.

Table 3: Screening of FinMCP-Bench (preliminary, heuristic; [TODO: replace with manually verified counts]).

	Multi-tool	Multi-turn
Tasks	249	219
Decision subset (≥ 1 decision)	149	63
choose one: tasks / median cand.	121 / 5	35 / 6
choose several: tasks / median cand.	84 / 20	33 / 10
Mechanical subset (control)	[TODO: 100]	[TODO: 156]
Mean LLM rounds per reference trajectory	5.73	5.00

RQ1 *Overall.* Is Jev-as-IF faster and cheaper than ReAct, plan-then-execute compilers and programmatic tool calling, without losing task success? Which error types does it reduce?

RQ2 *Graph guidance.* What do the split representation, derived data flow, scoped disclosure, typed stubs and the construction procedure each contribute?

RQ3 *System-1 conditionals.* What do in-program primitives, the System-1 backend, graph-based elaboration, batching and the gate each contribute?

RQ4 *Interaction and scaling.* On top of programmatic tool calling, are graph guidance and Jev-as-IF complementary? How do the savings scale with chain length, fan-out and the number of tools?

6.1 Data and Task Screening

FinMCP-Bench. Our main benchmark is FinMCP-Bench [37]: Chinese wealth-management requests over real MCP tools, with logged tool responses and reference answers. We use the multi-tool split (249 tasks, 51 tools) as the main testbed and the multi-turn split (219) as a secondary one. **[TODO:** tool schema source; replay environment and live-server sanity subset.]

Screening by decision structure. Compilation methods differ most on tasks that contain semantic decisions, so we screen every task by how its reference trajectory obtains each argument. A value counts as a *decision* if it was chosen among several values of the same upstream field, and as *mechanical* otherwise. Choosing all candidates counts as mechanical. Table 3 gives the resulting splits. We report results separately on the *decision* subset and on the *mechanical* control subset. The extracted decisions also serve as the calibration set for the gate (§4.5). **[TODO:** manual verification of screening; inter-annotator agreement.]

Advisory deployment. An anonymized SOG from the deployment in §5, with synthetic customer records. **[TODO:** size: entities, properties, operations; scenario count.]

Scaling suites. A generator over the advisory SOG varies chain length, candidate-set size, fan-out size and name ambiguity. A distractor generator adds 50–500 unrelated tools. FuncBenchGen [16] serves as a mechanical-only control.

Baselines. ReAct with native function calling [32]; ReWOO [31]; LLMCompiler with re-planning [12]; programmatic tool calling (PTC), in which the agent writes Python over the raw tool schemas in the same sandbox and returns to the model whenever it needs a judgment [4, 30]. All methods use the same agent and answerer

Table 4: Main results on FinMCP-Bench multi-tool, decision subset / mechanical subset. [TODO: fill]

Method	Succ.	LLM calls	Tokens	Cost	P50 lat.
ReAct	–	–	–	–	–
ReWOO	–	–	–	–	–
LLMCompiler	–	–	–	–	–
PTC	–	–	–	–	–
Jev-as-IF (ours)	–	–	–	–	–

Table 5: Error types per 100 tasks. [TODO: fill]

Method	Invalid call	Wrong entity	Fabricated argument	Logic / decision	Answer
ReAct	–	–	–	–	–
PTC	–	–	–	–	–
Jev-as-IF	–	–	–	–	–

Table 6: Graph ablation with the Jev-as-IF compiler fixed. All representations are rendered from the same curated source. [TODO: fill]

Representation	1st-pass	Dec. acc.	Succ.	Prompt tok.
Flat tool schemas	–	–	–	–
Operation graph only	–	–	–	–
Object graph only	–	–	–	–
Unified single graph	–	–	–	–
SOG w/o static checking	–	–	–	–
SOG (ours)	–	–	–	–

LLM, the same tool schemas and the same environment. **[TODO:** models; temperature; runs per configuration.]

Metrics. Task success (LLM judge validated against human labels); LLM calls on the critical path; LLM input and output tokens; System-1 tokens; cost at list prices; end-to-end latency (P50/P95); first-pass check rate (programs accepted without repair); decision accuracy; fallback rate. Errors are classified as invalid call, wrong entity, fabricated argument, program logic, decision, or answer.

6.2 RQ1: Overall Results

[TODO: Tables 4 and 5, with paired confidence intervals.]

6.3 RQ2: Graph Guidance

[TODO: Table 6. Disclosure: $h \in \{0, 1, 2, \infty\}$ vs. top-k tool retrieval, plotted against the number of tools. Construction: automatic vs. curated SOG (success, recall of trajectory dependencies by derived edges, curation person-hours).]

6.4 RQ3: System-1 Conditionals

[TODO: Table 7. Gate: realized wrong-action rate and fallback rate vs. α. Breakdown by primitive (choice-one, choice-several, noul guard, filter). Order debiasing on/off.]

Table 7: How semantic decisions are executed, on the decision subset, with the SOG fixed. [TODO: fill]

Variant	Succ.	LLM calls	Tokens	Lat.
No primitives (return to LLM)	–	–	–	–
No primitives (code rules only)	–	–	–	–
Re-plan at each decision	–	–	–	–
Primitives evaluated by LLM	–	–	–	–
Primitives evaluated by OSS model	–	–	–	–
Jev-as-IF w/o G_O elaboration	–	–	–	–
Jev-as-IF w/o batching	–	–	–	–
Jev-as-IF w/o gate	–	–	–	–
Jev-as-IF (ours)	–	–	–	–

Table 8: Factorial study on top of programmatic tool calling: graph guidance × System-1 primitives (success / LLM calls). [TODO: fill]

	No primitives	Jev-as-IF primitives
Flat tool schemas	– (= PTC)	–
SOG guidance	–	– (= ours)

6.5 RQ4: Interaction and Scaling

[TODO: Table 8 with main effects and interaction. Scaling plots: LLM calls and tokens vs. chain length and fan-out; prompt tokens and success vs. number of tools.]

7 Related Work

Code as action. CodeAct [30] and programmatic tool calling [4] let the model chain calls in code and keep intermediate results out of the context. AppWorld [27] is a demanding testbed for this style. These agents write code against raw tool schemas, nothing checks the code before it runs, and a semantic judgment mid-program sends control back to the model. Jev-as-IF keeps the medium but adds a graph-derived type system, static checking, and in-program System-1 primitives.

Plan-then-execute compilers. ReWOO decouples planning from observation with evidence variables [31]. LLMCompiler executes a task DAG in parallel and re-plans when needed [12]. Related lines fuse similar calls [22], plan with abstract placeholders that tools later fill [8], plan backwards from the target API [35], or train small planners [7]. These methods resolve mechanical dependencies at execution time and call the LLM back for everything else. DSPy compiles declarative pipelines of language-model calls [11], which optimizes prompts rather than tool chains.

Graph-guided tool use. Parameter-level API dependency graphs nearly double multi-tool performance, but they are annotated by experts or inferred by models [13]. TaskBench represents decomposed tasks as tool graphs [21]. Graph-based planners prune a tool dependency graph before planning [26] or learn to emit dependency-aware subtask graphs [25]. Dataflow synthesis treats dialogue state as an executable graph [20]. Dynamic ontologies have been proposed as an agent kernel for knowledge and reasoning [34]. These

works use a single graph to select or plan tools. Jev-as-IF uses a split graph to *compile*: the operation side types and checks the program, and the object side gives run-time decisions their semantics.

Systems for LLM applications. Parrot exposes application DAGs to the serving layer [15]. Asynchronous function calling overlaps generation with execution [10]. These systems shorten the time spent waiting for the LLM. Jev-as-IF removes the LLM from the execution loop except where the gate calls it back.

System-1 judgments, calibration and cascades. LLM cascades escalate from cheap to expensive models [6], and System-1 judges with escalation have been evaluated in [14]. Conformal prediction gives distribution-free coverage [1, 29], and conformal risk control extends it to general losses [2]. KnowNo asks for help when the prediction set is not a singleton [18]. Selective classification trades coverage for risk [9]. Agents also bind the wrong entity even with the right tool, and confidence-gated binding is one remedy [24]. Jev-as-IF applies these ideas to the conditionals of a compiled program.

Benchmarks. Nested and multi-hop tool benchmarks [5, 16, 17, 23, 33, 36] and financial MCP benchmarks [37] measure tool-use competence. We additionally screen tasks by whether they contain semantic decisions, because that is where compilation methods differ.

8 Discussion and Limitations

What "accurate" means here. Writing the chain once removes the agent's chance to react to each observation. We therefore claim no loss of task success rather than a gain. The gains we expect are specific: fewer invalid calls (static checking), fewer wrong-entity decisions (graph semantics), and no fabricated arguments (provenance). **[TODO:** confirm with Table 5.**]**

Graph authoring cost. Jev-as-IF needs a SOG. The construction procedure automates operation, property, entity and relation induction, but meanings, rules and access annotations still need an expert. The same artifact also serves as documentation, access policy and audit schema. **[TODO:** curation person-hours; quality gap between automatic and curated graphs.**]**

Confidently wrong decisions. The gate controls risk only if the probabilities are informative. System-1 models can be confidently misled by near-duplicate names or by text that argues for its own classification [14]. Calibration on in-distribution data does not protect against such shifts. **[TODO:** report the hard-case subset separately.**]**

Program errors. The gate catches uncertain decisions, not wrong programs. Static checking catches only structural errors. A program with sound structure but wrong logic fails silently until the answer is judged. **[TODO:** error attribution.**]**

Expressiveness. PyIF forbids unbounded loops and user-defined functions. Tasks that need open-ended exploration, such as searching until a condition holds, fall back to the agent loop. **[TODO:** share of FinMCP tasks expressible in PyIF.**]**

Closed models and language. Jev is proprietary and its documentation reports lower accuracy for CJK languages. We therefore report every result with an open-source backend as well.

Scope. We target read-mostly information tasks. Writes need idempotent protocols and compensation, which the runtime supports but our evaluation does not cover.

Data and ethics. FinMCP-Bench is released under CC BY-NC-SA 4.0, and our screened splits inherit this license. No customer data from the deployment is released; deployment scenarios use synthetic records. **[TODO:** GenAI usage disclosure per ACM policy.**]**

9 Conclusion

Web agents are slow and expensive because every dependent tool call costs an LLM round, and writing code removes only the mechanical rounds. Jev-as-IF is a graph-guided tool compiler that removes the semantic ones as well. A split object–operation graph gives the agent typed stubs, gives the compiler data-flow checks, and gives each run-time decision its entity semantics. System-1 primitives let the agent write every decision into the program, and a calibrated model evaluates them under a gate that calls the LLM back only when needed. The agent writes code once, and Jev evaluates its decisions. **[TODO:** one-sentence headline result.**]**

References

[1] Anastasios N. Angelopoulos and Stephen Bates. 2021. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. *arXiv preprint arXiv:2107.07511* (2021).

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations (ICLR)*.

[3] Anthropic. 2024. Model Context Protocol Specification. https://modelcontextprotocol.io.

[4] Anthropic. 2025. Introducing Advanced Tool Use on the Claude Developer Platform. https://www.anthropic.com/engineering/advanced-tool-use.

[5] Kinjal Basu et al. 2024. NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls. *arXiv preprint arXiv:2409.03797* (2024).

[6] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).

[7] Lutfi Eren Erdogan, Nicholas Lee, et al. 2024. TinyAgent: Function Calling at the Edge. In *Proceedings of EMNLP: System Demonstrations*. arXiv:2409.00608.

[8] Silin Gao, Jane Dwivedi-Yu, Ping Yu, et al. 2024. Efficient Tool Use with Chain-of-Abstraction Reasoning. *arXiv preprint arXiv:2401.17464* (2024).

[9] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[10] In Gim, Seung-seob Lee, and Lin Zhong. 2024. Asynchronous LLM Function Calling. *arXiv preprint arXiv:2412.07017* (2024).

[11] Omar Khattab et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations (ICLR)*.

[12] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine Learning (ICML)*.

[13] S. Lee, N. Kim, and Y. Jo. 2026. In-N-Out: A Parameter-Level API Graph Dataset for Tool Agents. *Transactions of the Association for Computational Linguistics* (2026). arXiv:2509.01560.

[14] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. 2026. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv preprint arXiv:2609.26550* (2026).

[15] Chaofan Lin et al. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[16] Seiji Maekawa et al. 2026. Towards Reliable Benchmarking: A Contamination Free, Controllable Evaluation Framework for Multi-step LLM Function Calling. In *International Conference on Learning Representations (ICLR)*. arXiv:2509.26553.

[17] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL). In *International Conference on Machine Learning (ICML)*.
[18] Allen Z. Ren et al. 2023. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *Conference on Robot Learning (CoRL)*.
[19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[20] Semantic Machines, Jacob Andreas, et al. 2020. Task-Oriented Dialogue as Dataflow Synthesis. *Transactions of the Association for Computational Linguistics* 8 (2020).
[21] Yongliang Shen et al. 2024. TaskBench: Benchmarking Large Language Models for Task Automation. In *NeurIPS Datasets and Benchmarks Track*.
[22] Simranjit Singh et al. 2024. An LLM-Tool Compiler for Fused Parallel Function Calling. *arXiv preprint arXiv:2405.17438* (2024).
[23] Yifan Song et al. 2023. RestGPT: Connecting Large Language Models with Real-World RESTful APIs. *arXiv preprint arXiv:2306.06624* (2023).
[24] Rahul Suresh Babu and Shashank Indukuri. 2026. Entity Binding Failures in Tool-Augmented Agents. *arXiv preprint arXiv:2606.30531* (2026).
[25] TODO. 2025. GAP: Graph-Based Agent Planning with Parallel Tool Use and Reinforcement Learning. *arXiv preprint arXiv:2510.25320* (2025).
[26] TODO. 2025. NaviAgent: Graph-Driven Bilevel Planning for Scalable Tool Orchestration. *arXiv preprint arXiv:2506.19500* (2025).
[27] Harsh Trivedi et al. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of ACL*. arXiv:2407.18901.
[28] TypeSafe AI. 2026. TypeSafe Documentation: System One Models and Jev. https://docs.typesafe.ai. Accessed 2026-09-30.
[29] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer.
[30] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *International Conference on Machine Learning (ICML)*.
[31] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. *arXiv preprint arXiv:2305.18323* (2023).
[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
[33] Junjie Ye et al. 2025. ToolHop: A Query-Driven Benchmark for Evaluating Large Language Models in Multi-Hop Tool Use. In *Proceedings of ACL*. arXiv:2501.02506.
[34] X. Zhang, Z. Sun, C. Yang, Y. Cui, L. Guo, and W. Hu. 2026. Toward Effective and Reliable LLM Agents via Dynamic Ontology. *arXiv preprint arXiv:2608.22974* (2026).
[35] Yinger Zhang et al. 2024. Reverse Chain: A Generic-Rule for LLMs to Master Multi-API Planning. In *Findings of the Association for Computational Linguistics: NAACL*. arXiv:2310.04474.
[36] Lucen Zhong et al. 2025. ComplexFuncBench: Exploring Multi-Step and Constrained Function Calling under Long-Context Scenario. *arXiv preprint arXiv:2501.10132* (2025).
[37] Jie Zhu, Yimin Tian, et al. 2026. FinMCP-Bench: Benchmarking LLM Agents for Real-World Financial Tool Use under the Model Context Protocol. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. arXiv:2603.24943.

A Stubs and a Program

Stubs rendered from a disclosed SOG (abridged; names and codes are synthetic):

```
class Portfolio:            # an advisory portfolio
    code: str               # identifier; label: name
    name: str               # display name
    type: str               # strategy type
class Announcement:
    fundCode: str
    title: str              # announcement title
    date: date

def read_scope() -> list[Portfolio]:
    """Portfolios the customer may ask about.
    Must run first."""
def read_holdings(code: str) -> list[Holding]: ...
def read_announcements(fundCode: str)
        -> list[Announcement]: ...
```

The program the agent writes for "Which of my steady portfolios had announcements that could explain this week's drop?":

```
ps = read_scope()
steady = jev.choice(
    "Which portfolios does the customer call "
    "'steady'?", ps, k="*")
hits = []
for p in steady:
    for h in read_holdings(code=p.code):
        news = read_announcements(fundCode=h.fundCode)
        hits += jev.filter(
            "Could this announcement explain a "
            "recent drop in net value?", news)
answer(portfolios=steady, announcements=hits)
```

The compiler checks that `code` and `fundCode` are outputs of earlier calls in the operation graph. Elaboration labels each portfolio with its name and type from the object graph, and adds the meanings of announcement fields as criteria. Both primitives become batched Noul questions. The loops are lowered to parallel `foreach` nodes. No LLM is called during execution unless the gate refuses.

B Screening Procedure

[TODO: Matching heuristic; false positives (e.g., pagination parameters); manual verification protocol; inter-annotator agreement.**]**

C SOG Construction Details

[TODO: Similarity thresholds for property alignment; entity induction algorithm; examples of automatic vs. curated graphs.**]**

D PyIF Grammar and Prompts

[TODO: Grammar of the allowed subset; agent, answerer, repair and baseline prompts.**]**